

732A94 Advanced R programming

Bonus lecture: data munging, machine learning, big data

Johan Alenlöv (Slides based on Leif Jonssons, Måns Magnusson, Arash Haratian and Krzysztof Bartoszek)

Linköpings Universitet

- Data munging
 - Tidy data, `dplyr`, `tidyr`
 - Regular expressions
- Machine learning
 - Supervised and unsupervised learning
 - Bias, variance and model selection
- Probability functions in R
- Big data

1. Each variable forms a column
2. Each observation forms a row
3. Each type of observational unit forms a table

Examples: `iris`, `faithful` and `gapminder`.

Why tidy?

80% of “big data” work is data munging

Analysis and visualization are based on tidy data

Performant code

Messy data $\longrightarrow$ Tidy data $\longrightarrow$ Analysis

The first arrow is where most of the time goes – and where most of the mistakes are made.

- Verbs for handling data
- Highly optimized C++ code (backend)
- Handles larger datasets in R (no copy-on-modify)

dplyr: some of the important functions

- `mutate()`: adds new columns that are functions of existing columns
- `transmute()`: same, but removes the existing ones
- `select()`: picks columns based on their names
- `filter()`: picks rows based on their values
- `group_by()` and `summarise()`: reduce multiple rows to a single summary
- `arrange()`: changes the ordering of the rows

dplyr: an example

```
library(dplyr)
iris |>
  filter(Sepal.Length > 5) |>
  mutate(ratio = Sepal.Length / Sepal.Width) |>
  group_by(Species) |>
  summarise(n = n(), mean_ratio = mean(ratio)) |>
  arrange(desc(mean_ratio))
```

```
## # A tibble: 3 x 3
##   Species      n mean_ratio
##   <fct>    <int>     <dbl>
## 1 virginica    49      2.24
## 2 versicolor  47      2.16
## 3 setosa      22      1.44
```

The pipe

```
x |> f(y)      # native pipe (R >= 4.1)
x %>% f(y)     # magrittr pipe, from dplyr
```

Both mean $f(x, y)$. The native `|>` is now preferred: it is base R, so you need no package for it.

`%>%` still has features `|>` lacks (the `.` placeholder, and it works with anonymous functions), so you will see both in the wild.

Another package for tidying data:

- **separate()**: separates a character column into multiple columns using a regular expression or numeric position
- **extract()**: creates new columns from a character column given a string or regex
- **pivot_wider()**: increases the number of columns by collecting information spread along the rows
- **pivot_longer()**: increases the number of rows by collecting information spread along the columns

Cheat sheet: <https://rstudio.github.io/cheatsheets/data-transformation.pdf>

Messy data: five common problems

1. Column headers are values, not variable names
2. Multiple variables are stored in one column
3. Variables are stored in both rows and columns
4. Multiple types of observational units are stored in the same table
5. A single observational unit is stored in multiple tables

Messy 1: column headers are values

The `who` dataset: the column names contain `new_{method of diagnosis}_{gender}{age group}`.

```
library(tidyr)
dim(who)
```

```
## [1] 7240    60
```

```
names(who)[5:10]
```

```
## [1] "new_sp_m014" "new_sp_m1524" "new_sp_m2534"
```

```
## [4] "new_sp_m3544" "new_sp_m4554" "new_sp_m5564"
```

Messy 1: the solution is `pivot_longer()`

```
long_who <- who |>  
  pivot_longer(cols = new_sp_m014:newrel_f65,  
               names_to = "column_info",  
               values_to = "cases",  
               values_drop_na = TRUE)
```

Messy 1: the result

```
dim(long_who)
```

```
## [1] 76046      6
```

```
head(long_who, 4)
```

```
## # A tibble: 4 x 6
```

##	country	iso2	iso3	year	column_info	cases
##	<chr>	<chr>	<chr>	<dbl>	<chr>	<dbl>
## 1	Afghanistan	AF	AFG	1997	new_sp_m014	0
## 2	Afghanistan	AF	AFG	1997	new_sp_m1524	10
## 3	Afghanistan	AF	AFG	1997	new_sp_m2534	6
## 4	Afghanistan	AF	AFG	1997	new_sp_m3544	3

Messy 2: multiple variables in one column

`column_info` contains three variables:

- method of diagnosis
- gender
- age group

Messy 2: the solution is `separate()`

```
tidy_who <- long_who |>
  # some observations use `newrel` not `new_rel`
  mutate(column_info = stringr::str_replace(
    column_info, "newrel", "new_rel")) |>
  separate(column_info, c("new", "method", "sexage")) |>
  select(-new, -iso2, -iso3) |>
  separate(sexage, c("sex", "age"), sep = 1)
```

`separate()` is superseded; the current spelling is `separate_wider_delim()` / `separate_wider_regex()`.

Messy 2: the result

```
head(tidy_who, 4)
```

```
## # A tibble: 4 x 6
##   country      year method sex   age  cases
##   <chr>      <dbl> <chr> <chr> <chr> <dbl>
## 1 Afghanistan 1997  sp    m    014     0
## 2 Afghanistan 1997  sp    m   1524    10
## 3 Afghanistan 1997  sp    m   2534     6
## 4 Afghanistan 1997  sp    m   3544     3
```

Messy 3: variables in both rows and columns

```
head(table2, 4)
```

```
## # A tibble: 4 x 4
##   country      year type      count
##   <chr>      <dbl> <chr>    <dbl>
## 1 Afghanistan 1999 cases      745
## 2 Afghanistan 1999 population 19987071
## 3 Afghanistan 2000 cases      2666
## 4 Afghanistan 2000 population 20595360
```

`count` means two different things depending on `type` – so `type` is a column header hiding in the data.

Messy 3: the solution is `pivot_wider()`

```
table2 |>  
  pivot_wider(names_from = type, values_from = count)
```

```
## # A tibble: 6 x 4  
##   country      year  cases population  
##   <chr>      <dbl> <dbl>      <dbl>  
## 1 Afghanistan 1999     745   19987071  
## 2 Afghanistan 2000    2666   20595360  
## 3 Brazil      1999   37737   172006362  
## 4 Brazil      2000   80488   174504898  
## 5 China       1999  212258  1272915272  
## 6 China       2000  213766  1280428583
```

A language for manipulating strings:

- Find strings that match a pattern
- Extract patterns from strings
- Replace patterns in strings

A component in many functions: `grep`, `gsub`, `stringr::`, `tidyr::`

Regular expressions: syntax

```
fruit <- c("apple", "banana", "pear", "pineapple")
```

Symbol	Description	Example
?	The preceding item is optional, matched at most once	<code>grep("pi?", fruit)</code>
*	The preceding item is matched zero or more times	<code>grep("pi*", fruit)</code>
+	The preceding item is matched one or more times	<code>grep("pi+", fruit)</code>
{n}	The preceding item is matched exactly n times	<code>grep("p{2}", fruit)</code>

Regular expressions: the rest of the alphabet

Pattern	Matches
.	any single character
[abc]	one of a, b, c
[^abc]	anything <i>but</i> a, b, c
[a-z], [0-9]	ranges
\\d, \\w, \\s	digit, word character, whitespace
^, \$	start / end of the string
(...)	a capture group
a b	a or b

In R you write "\\d" – one backslash for the regex, one to escape it in the R string.

Regex: finding matches

```
library(gapminder)
head(grep("we", gapminder$country), 10)
```

```
## [1] 1465 1466 1467 1468 1469 1470 1471 1472 1473 1474
```

```
unique(grep("we", gapminder$country, value = TRUE))
```

```
## [1] "Sweden" "Zimbabwe"
```

Regex: finding matches

```
grep("^New", state.name, value = TRUE)
```

```
## [1] "New Hampshire" "New Jersey"      "New Mexico"  
## [4] "New York"
```

```
sum(grepl("a$", state.name))      # ending in "a"
```

```
## [1] 21
```

Regex: the same with `stringr`

```
library(stringr)
str_subset(state.name, "^New")
```

```
## [1] "New Hampshire" "New Jersey"      "New Mexico"
## [4] "New York"
```

```
str_detect(state.name, "a$") |> sum()
```

```
## [1] 21
```

```
str_which(state.name, "^New")
```

```
## [1] 29 30 31 32
```

`stringr` functions all start with `str_`, take the string first and the pattern second – which is what makes them pipe-friendly.

Regex: extraction

```
x <- c("The 13 Cats in the Hats are 17 years",  
      "4 scores and 7 beers ago")  
str_extract(x, "[a-z]+")      # first lower-case word
```

```
## [1] "he"      "scores"
```

```
str_extract_all(x, "[0-9]+") # all runs of digits
```

```
## [[1]]
```

```
## [1] "13" "17"
```

```
##
```

```
## [[2]]
```

```
## [1] "4" "7"
```

Regex: a more complicated pattern

```
# the starting word of the first 6 sentences  
str_extract(sentences[1:6], "[A-Z]+[a-z]*")
```

```
## [1] "The"    "Glue"   "It"     "These"  "Rice"   "The"
```

```
# article + noun  
noun <- "(a|the) ([^ ]+)"  
str_match(sentences[1:4], noun)
```

```
##      [,1]      [,2] [,3]  
## [1,] "the smooth" "the" "smooth"  
## [2,] "the sheet"  "the" "sheet"  
## [3,] "the depth"  "the" "depth"  
## [4,] "a chicken"  "a"   "chicken"
```

```
head(table3, 3)
```

```
## # A tibble: 3 x 3
##   country      year rate
##   <chr>      <dbl> <chr>
## 1 Afghanistan 1999 745/19987071
## 2 Afghanistan 2000 2666/20595360
## 3 Brazil      1999 37737/172006362
```

The **rate** column holds two variables separated by `/`.

Regex: `tidyr::separate()`

```
table3 |>  
  separate(rate, c("cases", "pop"), sep = "/") |>  
  head(3)
```

```
## # A tibble: 3 x 4  
##   country      year cases pop  
##   <chr>      <dbl> <chr> <chr>  
## 1 Afghanistan 1999  745  19987071  
## 2 Afghanistan 2000 2666  20595360  
## 3 Brazil      1999 37737 172006362
```

Regex: `tidyr::extract()`

```
tibble(sentence = sentences[1:5]) |>
  extract(sentence, c("article", "noun"), noun,
          remove = FALSE)
```

```
## # A tibble: 5 x 3
```

##	sentence	article	noun
##	<chr>	<chr>	<chr>
## 1	The birch canoe slid on the smooth planks.	the	smooth
## 2	Glue the sheet to the dark blue background.	the	sheet
## 3	It's easy to tell the depth of a well.	the	depth
## 4	These days a chicken leg is a rare dish.	a	chicken
## 5	Rice is often served in round bowls.	<NA>	<NA>

`extract()` uses the *capture groups* of the regex as the new columns.

- Build the pattern up piece by piece, testing as you go
- `str_view()` highlights what matched – use it constantly
- Anchor (^, \$) whenever you can; unanchored patterns match more than you think
- `fixed()` when you want a literal string, not a pattern
- Regex is write-only. Comment it, or use `regex(..., comments = TRUE)`

<https://regex101.com/> is worth the visit.

- Automatically detect patterns in data
- Predict future observations
- Decision making under uncertainty

- Supervised learning
- Unsupervised learning
- Reinforcement learning

(also called predictive learning)

- response variable
- covariates / features
- training set

$$\mathcal{D} = (x_i, y_i)_{i=1}^N$$

If y_i is categorical: **classification**

If y_i is real: **regression**

(also called knowledge discovery)

$$\mathcal{D} = (x_i)_{i=1}^N$$

- dimensionality reduction
- latent variable modeling
- clustering, PCA, discovering graph structures
- data visualization

The more variables, the larger the distance between data points.

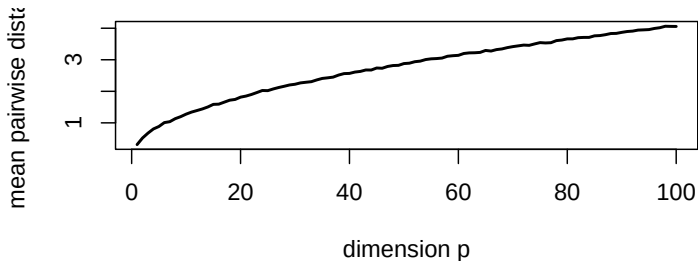
Euclidean metric:

$$d_2(\vec{x}, \vec{y}) = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2 + \cdots + (x_p - y_p)^2}$$

Everything is far from everything else, and “nearest neighbour” stops meaning anything.

Curse of dimensionality

```
d <- sapply(1:100, \(k) mean(dist(matrix(runif(200 * k),  
                                         200, k))))  
plot(1:100, d, type = "l", xlab = "dimension p")
```



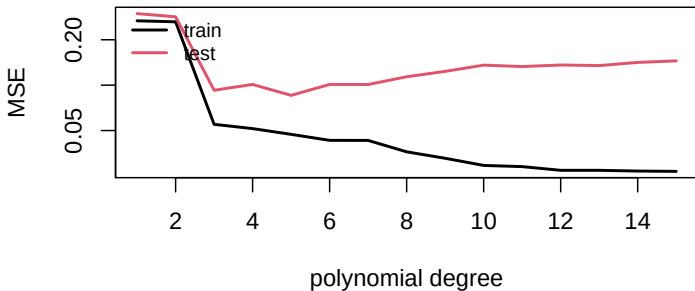
Underfit = bad fit, low variance

Overfit = good fit (to the training data), high variance

The model that fits your training data best is almost never the model that predicts best.

```
set.seed(1); n <- 30
x <- sort(runif(n)); y <- sin(2*pi*x) + rnorm(n, sd = .3)
xt <- sort(runif(n)); yt <- sin(2*pi*xt) + rnorm(n, sd = .3)
err <- sapply(1:15, function(d) {
  m <- lm(y ~ poly(x, d))
  c(train = mean(resid(m)^2),
    test = mean((yt - predict(m, data.frame(x = xt)))^2))
})
```

Bias and variance



The training error keeps falling; the test error turns around. That turning point is the model you want.

- Fit and variance – trade-off
- Hyperparameters
- Generalization error
- Validation set / cross validation
- Information criteria: model fit penalized for model dimensionality (AIC, BIC)

Predictive modeling pipeline

1. Set aside data for **test** (to estimate the generalization error)
2. Set aside data for **validation** (if there are hyperparameters)
3. Run algorithms
4. Find the best/optimal hyperparameters (on the validation set)
5. Choose the final model
6. Estimate the generalization error on the **test** set

Step 6 uses data that was never touched in steps 3–5. Break that and your error estimate is meaningless.

```
set.seed(1)
k <- 5
fold <- sample(rep(1:k, length.out = nrow(iris)))
cv <- sapply(1:k, function(i) {
  m <- lm(Sepal.Length ~ ., data = iris[fold != i, ])
  p <- predict(m, iris[fold == i, ])
  mean((iris$Sepal.Length[fold == i] - p)^2)
})
c(mse = mean(cv), se = sd(cv) / sqrt(k))
```

```
##          mse          se
## 0.09589308 0.01088432
```

- Different models work in different domains
- Accuracy–complexity–interpretability trade-off
- ... but more data almost always wins

The caret package

- A package for supervised learning
- Does not contain the methods – it is a **framework**
- Compare methods on hold-out data
- <http://topepo.github.io/caret/>
- The specific algorithms are part of other courses

caret: a typical call

```
library(caret)
set.seed(1)
ctrl <- trainControl(method = "cv", number = 5)
fit <- train(Species ~ ., data = iris,
             method = "rf", trControl = ctrl)
fit$results
```

```
##   mtry  Accuracy Kappa AccuracySD   KappaSD
## 1     2 0.9533333 0.93 0.01825742 0.02738613
## 2     3 0.9533333 0.93 0.01825742 0.02738613
## 3     4 0.9533333 0.93 0.01825742 0.02738613
```

One interface, hundreds of methods – `method = "rf", "glmnet", "svmRadial", ...` and `caret` does the resampling for you.

caret: what came out

```
fit$bestTune
```

```
##      mtry
```

```
## 1      2
```

```
predict(fit, newdata = iris[c(1, 60, 120), ])
```

```
## [1] setosa      versicolor virginica
```

```
## Levels: setosa versicolor virginica
```

tidymodels (parsnip + recipes + rsample + tune) is the actively developed successor; caret is in maintenance mode.

Prefix	Description	Example
r	Random draw	<code>rnorm</code>
d	Density function	<code>dbinom</code>
q	Quantile function	<code>qbeta</code>
p	CDF	<code>pgamma</code>

```
set.seed(1)
c(rnorm(1), dnorm(0), qnorm(0.975), pnorm(1.96))
```

```
## [1] -0.6264538  0.3989423  1.9599640  0.9750021
```

- whole genome sequencing
- surveillance cameras (CCTV)
- Internet traffic
- credit card transactions
- everything communicating with everything

... to computational complexity. What is “too big”?

$$O(N) \quad N \approx 10^{12}$$

$$O(N^2) \quad N \approx 10^6$$

$$O(N^3) \quad N \approx 10^4$$

$$O(2^N) \quad N \approx 50$$

We need algorithms that scale!

Complexity of some common methods

Complexity	Method
$O(P^2 N)$	Linear regression
$O(N^3)$	Gaussian processes
$O(N^2) / O(N^3)$	Support vector machines
$O(T \cdot PN \log N)$	Random forests
$O(I \cdot N)$	Topic models

The exponent on N is what decides whether your method survives contact with the data.

R stores data in RAM.

Type	Size
integers	4 bytes
numerics	8 bytes

A matrix with 100M rows and 5 numeric columns:

$$100\,000\,000 \times 5 \times 8 / 1024^3 \approx 3.8 \text{ GB}$$

See `help(Memory)` and `help("Memory-limits")`.

Genome storage...?

Measuring it yourself

```
x <- numeric(1e6)
format(object.size(x), units = "MB")
```

```
## [1] "7.6 Mb"
```

```
y <- integer(1e6)
format(object.size(y), units = "MB")
```

```
## [1] "3.8 Mb"
```

```
format(object.size(iris), units = "KB")
```

```
## [1] "7.1 Kb"
```

`gc()` tells you how much R currently holds.

How to deal with large data sets

- Handle chunkwise
- Subsampling
- More hardware
- C++/Java backend (`dplyr`, `data.table`)
- Reduce the data in memory (right types, factors, sparse matrices)
- Database backend (`DBI` + `dbplyr`, `duckdb`, `arrow`)

Spark and **sparklyr** – fast cluster computations for machine learning and statistics.

arrow and **duckdb** handle larger-than-memory data on a single machine, and both work behind **dplyr** verbs – often that is all you need.

The End... for the meantime

Questions?

Good luck!